SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Data Interpretation

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO4 – Precision	**Assessment:**	Assessment Tool 1 – Data Interpretation
Weightage:	35% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	G. Jaswanth	**Reg. No.:**	192472235
Section / Batch:	2024	**Date:**	

Code of Conduct

I G. Jaswanth - 192472235 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *G. Jaswanth*

Instructions

- Read each data set, table, semantic representation, or scenario carefully before answering.
- Analyze the given information and provide logical, evidence-based interpretations.
- Show all intermediate reasoning, semantic analysis steps, predicate representations, or calculations wherever applicable.
- Answers should be concise, structured, and technically accurate.
- Support your conclusions using the data provided in the question.
- Draw diagrams, semantic networks, parse trees, or logical representations wherever relevant.
- Avoid copying definitions alone; focus on analyzing the given scenario and interpreting the data.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic representations, identify action–entity relationships, detect interpretation errors, and evaluate meaning representation in chatbot systems.	Q1
First-Order Predicate Calculus & Representing Linguistically Relevant Concepts	Construct predicate representations, apply logical inference rules, derive conclusions, and evaluate reasoning in smart manufacturing environments.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze contextual clues, determine correct word senses, evaluate ambiguity resolution, and improve semantic understanding in search systems.	Q3
Syntax-Driven Semantic Analysis	Analyze syntactic structures, assign semantic roles, evaluate parsing accuracy, and improve semantic interpretation in healthcare NLP applications.	Q4

Annexure A – Data Interpretation

1. Frame Semantics in Banking Customer-Service Systems

A digital banking platform uses semantic frames to understand customer requests. The NLP system identifies the **action, agent, object, and destination** involved in each request.

Customer Query	Generated Semantic Frame
"Transfer ₹20,000 from my savings account to my son's account."	TRANSFER(SourceAccount, Amount, DestinationAccount, Customer)
"Block my debit card immediately."	BLOCK(DebitCard, Customer)
"Send my transaction statement to my email."	SEND(TransactionStatement, Email, Customer)
"Increase my daily withdrawal limit."	MODIFY(WithdrawalLimit, Customer)

Additional transaction logs show:

Query ID	Actual User Intent	System Interpretation
B1	Transfer money to another account	Transfer money
B2	Block debit card	Block debit card
B3	Receive transaction statement by email	Send statement to email
B4	Increase withdrawal limit	Decrease withdrawal limit

Tasks:

1. Analyze the semantic frame of each banking request and identify the relationship between the action and its participants.
2. Identify the query in which the semantic interpretation is incorrect and explain why.
3. Analyze how incorrect identification of semantic roles can lead to an incorrect banking operation.
4. Recommend a frame-based semantic analysis approach to improve the reliability of banking conversational systems.

2. First-Order Predicate Calculus for Smart Agriculture

A smart agricultural system monitors crop fields using sensors and logical rules.

Field Data:

Field	Soil Condition	Irrigation Status	Crop
F1	Dry	Available	Rice
F2	Wet	Available	Wheat
F3	Dry	Unavailable	Maize
F4	Wet	Unavailable	Rice

The system uses the following predicate rules:

- $Dry(x) \land IrrigationAvailable(x) \rightarrow NeedsWater(x)$
- $NeedsWater(x) \rightarrow Irrigate(x)$
- $Wet(x) \rightarrow \neg NeedsWater(x)$
- $IrrigationUnavailable(x) \rightarrow \neg Irrigate(x)$
- $Rice(x) \land NeedsWater(x) \rightarrow PriorityIrrigation(x)$

Tasks:

1. Represent the field observations using First-Order Predicate Calculus.
2. Using the given rules, determine which fields require irrigation.
3. Determine whether F1 and F3 should be treated differently by the automated irrigation controller.
4. Analyze whether predicate logic alone is sufficient for agricultural decision support when sensor information is incomplete or uncertain.

3. Word Sense Disambiguation in a Travel Recommendation System

A travel platform receives search queries containing ambiguous words. The recommendation engine uses surrounding words and user interactions to determine the intended meaning.

Search Query	Possible Sense 1	Possible Sense 2
"Amazon tour"	Amazon rainforest	Amazon company
"Safari booking"	Wildlife tour	Web browser
"Java vacation"	Java island	Java programming language
"Cruise package"	Ship journey	Cruise software/project

The system records the following user interactions:

Query	User Interaction
Amazon tour	Viewed rainforest trekking packages
Safari booking	Selected wildlife resort packages
Java vacation	Viewed hotels in Bali and Java
Cruise package	Viewed Mediterranean ship packages

However, another user searches: **"Safari download for Windows"** and clicks a browser download page.

Tasks:

1. Determine the intended sense of the ambiguous terms using the available context.
2. Identify the lexical and contextual cues that distinguish the possible senses.
3. Analyze why the same word may require different interpretations for different users.
4. Design an industrial-scale WSD strategy using query context, user history, embeddings, and click behaviour to improve travel recommendations.

4. Syntax-Driven Semantic Analysis in Legal Document Processing

A legal NLP system analyzes sentences from contracts and assigns semantic roles based on their syntactic structures.

The system produces the following analyses:

Legal Sentence	Parse Structure	Generated Semantic Interpretation
"The supplier delivered the equipment to the buyer."	Subject–Verb–Object	Supplier = Agent, Equipment = Object, Buyer = Recipient
"The buyer received the equipment from the supplier."	Subject–Verb–Object	Buyer = Agent, Equipment = Object, Supplier = Recipient
"The company terminated the contract after the violation."	Subject–Verb–Object	Company = Agent, Contract = Object
"The contract was terminated by the company."	Passive Construction	Contract = Agent, Company = Object

Tasks:

1. Analyze how syntactic structure, including active and passive constructions, influences semantic-role assignment.
2. Identify the incorrect semantic-role assignments in the given interpretations.
3. Explain how confusing grammatical subject with semantic agent can affect automated legal information extraction.
4. Recommend a syntax-driven semantic analysis architecture that can correctly handle active voice, passive voice, prepositional phrases, and long-distance dependencies in legal documents.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation

		structure, visuals, and terminology			and difficult to understand

Q. No. / Criteria Level	Understanding of Data	Analysis	Application of Concepts	Conclusion	Clarity	Sub. Total	Total %
1	4	4	4	3	3	18	92
2	4	4	3	4	4	19	
3	4	4	3	3	4	18	
4	4	4	3	3	3	17	

Faculty In-charge	Course Coordinator	Program Director
Signature: ________________	Signature: ________________	Signature: ________________
Date: ____________________	Date: ____________________	Date: ____________________

Q1. Frame Semantics in Banking Customer-Service Systems

1. Analyze the semantic frame of each banking request

A semantic frame represents the **main action** and the participants involved in that action. In the given banking system, the important roles are the **action, agent/customer, object, source, destination, or target**.

Customer Query	Semantic Frame	Analysis
“Transfer ₹20,000 from my savings account to my son's account.”	TRANSFER(SourceAccount, Amount, DestinationAccount, Customer)	**Customer** is the agent. The **savings account** is the source, ₹20,000 is the amount, and the **son's account** is the destination.
“Block my debit card immediately.”	BLOCK(DebitCard, Customer)	The **customer** requests the action, and the **debit card** is the object being blocked.
“Send my transaction statement to my email.”	SEND(TransactionStatement, Email, Customer)	The **transaction statement** is the object, the **email** is the destination, and the **customer** is the requester.
“Increase my daily withdrawal limit.”	MODIFY(WithdrawalLimit, Customer)	The action is **MODIFY**, the target is the **withdrawal limit**, and the customer requests the modification.

Semantic relationships

For example:

```
Customer
  |
  | requests TRANSFER
  ↓
₹20,000
  |
  ├── Source → Savings Account
  |
  └── Destination → Son's Account
```

Thus, frame semantics provides more information than simply identifying the main verb. It identifies **who performs or requests the action and what entities participate in it**.

2. Identify the incorrect semantic interpretation

The incorrect interpretation is:

B4

Query ID	Actual User Intent	System Interpretation

Query ID	Actual User Intent	System Interpretation
B4	Increase withdrawal limit	**Decrease withdrawal limit**

The system has incorrectly interpreted the user's requested operation.

The difference is significant:

Increase =Decrease

The intended operation is:

MODIFY(WithdrawalLimit, Customer)

with the modification direction being **increase**.

The system instead interprets it as a **decrease**, which reverses the intended meaning.

Other interpretations

- **B1:** Transfer money → correctly interpreted as transfer money.
- **B2:** Block debit card → correctly interpreted.
- **B3:** Receive transaction statement by email → correctly interpreted as sending the statement to email.
- **B4: Incorrect**, because increase was interpreted as decrease.

Therefore:

B4 is the incorrect semantic interpretation

3. Effect of incorrect semantic-role identification

Incorrect semantic-role identification can cause a banking system to perform the **wrong financial operation**.

Consider B4:

> "Increase my daily withdrawal limit."

The intended operation is:

MODIFY
↓
Withdrawal Limit
↓
Increase

But the system interprets it as:

MODIFY
↓
Withdrawal Limit
↓
Decrease

This is not a minor interpretation error. It changes the **direction of the requested operation**.

Possible consequences

1. **Incorrect account modification**
   The customer's withdrawal limit could be reduced instead of increased.
2. **Customer inconvenience**
   The customer may be unable to withdraw the required amount.
3. **Financial risk**
   An incorrect automated banking action could affect the customer's ability to access funds.
4. **Loss of trust**
   Customers may lose confidence in an automated banking assistant if it performs actions different from their requests.
5. **Need for confirmation**
   High-impact banking operations should ideally be verified before execution when the semantic interpretation is uncertain.

Why semantic roles matter

A banking system should not only identify:

> "modify withdrawal limit"

It must also correctly identify **what modification is requested**.

Therefore, semantic analysis should preserve important attributes such as:

Action → MODIFY
Object → WithdrawalLimit
Direction → INCREASE
Agent → Customer

4. Recommended Frame-Based Semantic Analysis Approach

A reliable banking conversational system should use a **frame-based semantic analysis pipeline**.

Recommended architecture

Customer Query
↓
Text Preprocessing
↓
Intent Detection
↓
Semantic Frame Identification
↓
Role / Slot Extraction
↓
Semantic Validation
↓
Confidence Check
↓
Banking Operation

Step 1 – Intent Detection

Identify the main action:

transfer
block
send
modify

For example:

> "Increase my daily withdrawal limit."

becomes:

```
Intent = MODIFY
```

Step 2 – Frame Construction

Construct a structured representation:

```
MODIFY(
   Object = WithdrawalLimit,
   Customer = User,
   Direction = Increase
)
```

Step 3 – Semantic Role Extraction

Identify the participants and attributes:

Role	Value
Agent	Customer
Action	Modify
Object	Withdrawal Limit
Direction	Increase

Step 4 – Semantic Validation

The system should check whether the extracted meaning is internally consistent.

For example:

```
User says: Increase
System detects: Decrease
         ↓
     CONFLICT
         ↓
Ask for confirmation
```

Step 5 – Confidence-Based Confirmation

For high-risk banking actions, the system should ask for confirmation when confidence is low.

For example:

> "You want to **increase your daily withdrawal limit**. Should I proceed?"

This prevents an incorrect semantic interpretation from directly becoming a banking transaction.

Conclusion

The given banking examples demonstrate that **semantic frames are essential for understanding customer requests beyond individual words**. B1, B2, and B3 are interpreted consistently, whereas **B4 contains a critical semantic error: "increase" is interpreted as "decrease."**

A frame-based system that combines **intent detection, semantic-role extraction, validation, confidence checking, and confirmation for uncertain high-impact operations** would make banking conversational systems more reliable and safer.

Q2. First-Order Predicate Calculus for Smart Agriculture

1. Represent the Field Observations Using First-Order Predicate Calculus

The field data can be represented using predicates for **soil condition, irrigation availability, and crop type**.

Field F1

Given:

- Soil = Dry
- Irrigation = Available
- Crop = Rice

Therefore:

Dry(F1) IrrigationAvailable(F1) Rice(F1)

Field F2

Given:

- Soil = Wet
- Irrigation = Available
- Crop = Wheat

Therefore:

Wet(F2) IrrigationAvailable(F2) Wheat(F2)

Field F3

Given:

- Soil = Dry
- Irrigation = Unavailable
- Crop = Maize

Therefore:

Dry(F3) IrrigationUnavailable(F3) Maize(F3)

Field F4

Given:

- Soil = Wet
- Irrigation = Unavailable
- Crop = Rice

Therefore:

$Wet(F4)\ IrrigationUnavailable(F4)\ Rice(F4)$

Complete representation

$Dry(F1) \land IrrigationAvailable(F1) \land Rice(F1) Wet(F2) \land IrrigationAvailable(F2) \land Wheat(F2) Dry(F3) \land IrrigationUnavailable(F3) \land Maize(F3) Wet(F4) \land IrrigationUnavailable(F4) \land Rice(F4)$

2. Determine Which Fields Require Irrigation

The given rules are:

Rule 1

$Dry(x) \land IrrigationAvailable(x) \rightarrow NeedsWater(x)$

Rule 2

$NeedsWater(x) \rightarrow Irrigate(x)$

Rule 3

$Wet(x) \rightarrow \neg NeedsWater(x)$

Rule 4

$IrrigationUnavailable(x) \rightarrow \neg Irrigate(x)$

Rule 5

$Rice(x) \land NeedsWater(x) \rightarrow PriorityIrrigation(x)$

Now apply the rules to each field.

F1

Given:

$Dry(F1)$

and:

$IrrigationAvailable(F1)$

Using Rule 1:

$Dry(F1) \land IrrigationAvailable(F1) \rightarrow NeedsWater(F1)$

Therefore:

$NeedsWater(F1)$

Using Rule 2:

$NeedsWater(F1) \rightarrow Irrigate(F1)$

Therefore:

Irrigate(F1)

Since F1 is rice:

Rice(F1)∧NeedsWater(F1)→PriorityIrrigation(F1)

Therefore:

PriorityIrrigation(F1)

F2

Given:

Wet(F2)

Using Rule 3:

Wet(F2)→¬NeedsWater(F2)

Therefore:

¬NeedsWater(F2)

Hence F2 does not require irrigation.

F3

Given:

Dry(F3)

but:

IrrigationUnavailable(F3)

Rule 1 requires **both** dry soil and available irrigation.

Therefore:

Dry(F3)∧IrrigationAvailable(F3)

cannot be established.

So we cannot derive:

NeedsWater(F3)

Furthermore, Rule 4 directly gives:

IrrigationUnavailable(F3)→¬Irrigate(F3)

Therefore:

¬Irrigate(F3)

F3 is dry, but irrigation cannot be performed because irrigation is unavailable.

F4

Given:

Wet(F4)

Using Rule 3:

Wet(F4)→¬NeedsWater(F4)

Therefore:

¬NeedsWater(F4)

Also:

IrrigationUnavailable(F4)→¬Irrigate(F4)

Therefore:

¬Irrigate(F4)

Final decision table

Field	Soil	Irrigation	Needs Water	Irrigate?	Priority
F1	Dry	Available	Yes	**Yes**	**Yes**
F2	Wet	Available	No	No	No
F3	Dry	Unavailable	Cannot derive	**No**	No
F4	Wet	Unavailable	No	**No**	No

Conclusion

Under the **given rules**, only **F1 should be irrigated**.

F1→Irrigate

F3 requires special attention because it is dry, but irrigation is unavailable.

3. Should F1 and F3 Be Treated Differently?

Yes.

Although both fields have **dry soil**, their irrigation conditions are different.

F1
Dry(F1) IrrigationAvailable(F1)

Therefore:

NeedsWater(F1)

and:

Irrigate(F1)

Since F1 contains rice:

PriorityIrrigation(F1)

Thus F1 should be **irrigated immediately and given priority**.

F3

Dry(F3)

but:

IrrigationUnavailable(F3)

Therefore, Rule 4 gives:

¬Irrigate(F3)

Thus F3 **needs water based on its dry condition**, but the controller cannot perform irrigation because the required resource is unavailable.

Comparison

F1:
Dry + Available
↓
NeedsWater
↓
Irrigate
↓
PriorityIrrigation

F3:
Dry + Unavailable
↓
Cannot perform irrigation
↓
¬Irrigate

Therefore:

F1 and F3 must be treated differently

The important distinction is **resource availability**, not simply soil condition.

4. Is Predicate Logic Alone Sufficient?

No. Predicate logic alone is not sufficient for agricultural decision support when sensor information is incomplete or uncertain.

The given rules operate on definite facts such as:

Dry(F1)

or:

Wet(F2)

This works well when sensor information is accurate and complete.

However, real agricultural sensors can produce uncertain or incomplete information.

Example

Suppose a soil sensor reports:

Soil moisture = uncertain

Predicate logic requires a definite statement such as:

Dry(F1)

or:

Wet(F1)

It does not naturally represent the degree of confidence associated with the measurement.

Similarly, if the irrigation sensor temporarily fails, the system may not know whether:

IrrigationAvailable(F1)

is true or false.

Limitations

Predicate logic alone does not naturally handle:

- Sensor noise
- Missing sensor readings
- Measurement uncertainty
- Conflicting sensor observations
- Degrees of confidence
- Probabilistic environmental conditions

Better approach

A practical agricultural decision-support system could combine predicate rules with:

- **Probabilistic reasoning** for uncertain sensor data
- **Fuzzy logic** for gradual conditions such as "slightly dry" or "very dry"

- **Sensor confidence values**
- **Machine-learning predictions**
- **Real-time sensor validation**

For example:

Sensor reading
↓
Confidence estimation
↓
Fuzzy / probabilistic reasoning
↓
Predicate rules
↓
Irrigation decision

Final Conclusion

Predicate logic is useful for representing **clear facts and deterministic rules**, but it should not be the only reasoning mechanism when agricultural data is incomplete or uncertain.

For this scenario:

Predicate logic is useful but insufficient by itself

A hybrid system combining logical rules with **probabilistic or fuzzy reasoning** would provide more reliable agricultural decision support.

Q3. Word Sense Disambiguation in a Travel Recommendation System

Word Sense Disambiguation (WSD) determines the **intended meaning of an ambiguous word from its surrounding context and user behavior**.

1. Determine the Intended Sense

The available queries and user interactions provide strong evidence for selecting the intended meaning.

Search Query	Possible Senses	User Interaction	Intended Sense
Amazon tour	Amazon rainforest / Amazon company	Viewed rainforest trekking packages	**Amazon rainforest**
Safari booking	Wildlife tour / Web browser	Selected wildlife resort packages	**Wildlife tour**
Java vacation	Java island / Java programming language	Viewed hotels in Bali and Java	**Java island**
Cruise package	Ship journey / Cruise software/project	Viewed Mediterranean ship packages	**Ship journey / cruise**
Safari download for	Wildlife tour / Web browser	Clicked a browser download	**Web browser**

Search Query	Possible Senses	User Interaction	Intended Sense
Windows		page	

Interpretation

For the first four queries, the user's interactions confirm the travel-related sense:

- **Amazon tour → Amazon rainforest**
- **Safari booking → Wildlife safari**
- **Java vacation → Java island**
- **Cruise package → Ship journey**

The additional query demonstrates that the same word can have a completely different sense:

> **"Safari download for Windows"**

Here, **Safari** refers to the **web browser**, not a wildlife tour, because *download* and *Windows* are strong technology-related contextual clues.

Therefore:

WSD must consider both query context and user behavior

2. Lexical and Contextual Cues

The system can distinguish the senses using **lexical clues** and **contextual/user-behavior clues**.

Amazon

Travel sense:

Amazon + tour
rainforest
trekking
packages

These words strongly indicate the **Amazon rainforest**.

Company sense would be more likely with terms such as:

Amazon + shopping
delivery
Prime
products

Safari

Wildlife sense:

Safari + booking
wildlife
resort
tour

The interaction:

Selected wildlife resort packages

strongly confirms the wildlife interpretation.

Browser sense:

Safari + download
Windows
browser
software

The query:

Safari download for Windows

contains strong technology-related clues.

Java

Travel sense:

Java + vacation
hotels
Bali
travel

The user's interaction with hotels in Bali and Java strongly indicates **Java island**.

Programming sense would be associated with:

Java + programming
code
JDK
compiler
software

Cruise

Travel sense:

Cruise + package
Mediterranean
ship
journey

The user's interaction with Mediterranean ship packages confirms the travel meaning.

Software sense would be more likely with:

Cruise + software
project
development
Technology

Summary

Term	Travel/Non-Travel Cue

Term	Travel/Non-Travel Cue
Amazon	tour, rainforest, trekking
Safari	booking, wildlife, resort
Java	vacation, hotels, Bali
Cruise	package, Mediterranean, ship
Safari	download, Windows → browser

Thus, WSD should combine **the ambiguous word with nearby contextual words and behavioral evidence**.

3. Why the Same Word Has Different Interpretations

A word can have multiple senses because natural language is **context-dependent**.

For example:

Safari

Safari booking
↓
Wildlife context
↓
Safari = Wildlife tour

But:

Safari download for Windows
↓
Technology context
↓
Safari = Web browser

The word itself has not changed.

What changes is the **context surrounding it**.

Similarly:

Java vacation
↓
Travel context
↓
Java = Island

Java programming
↓
Technology context
↓
Java = Programming language

User-specific interpretation

Different users may also have different interests and histories.

For example, if one user frequently:

- searches hotels,
- views travel packages,
- books flights,

then a query such as:

Java

is more likely to represent the travel destination.

Another user who frequently searches:

- programming tutorials,
- JDK documentation,
- coding courses,

may intend **Java programming language**.

Therefore:

$$\text{Meaning}=f(\text{word},\text{context},\text{user history},\text{behavior})$$

This is why a travel recommendation system should not rely only on the ambiguous word itself.

4. Industrial-Scale WSD Strategy

A large-scale travel recommendation system should combine **query context, user history, embeddings, and click behavior**.

Proposed Architecture

User Query
↓
Text Preprocessing
↓
Context Extraction
↓

↓ Query Context ↓	↓ User History ↓	↓ Click Data ↓

↓
Embedding Model
↓
Candidate Senses
↓
Sense Scoring
↓
Best Sense Selection
↓
Travel Recommendation

Step 1 – Query Context

Extract words surrounding the ambiguous term.

For:

Safari download for Windows

the context is:

```
download
Windows
```

This strongly favors the technology/browser sense.

Step 2 – User History

Use previous interactions to understand the user's interests.

For example:

```
User history:
Hotels → Flights → Resorts → Java hotels
```

This increases the likelihood that:

```
Java → Java island
```

Step 3 – Embeddings

Represent the query and possible senses as numerical vectors using a language embedding model.

For example:

```
"Java vacation"
        ↓
Query embedding
        ↓
Compare with:
"Java island"
"Java programming language"
```

The sense with the higher semantic similarity becomes more likely.

Step 4 – Click Behaviour

User clicks provide valuable feedback.

For example:

```
Query: Amazon tour

Result A → Amazon rainforest
Result B → Amazon company

User clicks Result A
        ↓
Strong evidence
```

↓
Amazon = rainforest

Click behavior can therefore be used as an additional signal for WSD.

Step 5 – Sense Scoring

The system can combine multiple signals:

Score(s)=w1 Context+w2 History+w3 Embedding+w4 Clicks

where:

- **Context** = words surrounding the ambiguous term
- **History** = user's previous activity
- **Embedding** = semantic similarity
- **Clicks** = observed user preference

The sense with the highest score is selected.

Step 6 – Continuous Learning

The system can continuously learn from new searches and clicks.

For example:

Initial prediction
↓
User clicks result
↓
Feedback collected
↓
Sense model updated
↓
Future predictions improved

This allows the WSD system to adapt to changing user interests and new language patterns.

Conclusion

The given data demonstrates that **the same lexical item can have different meanings depending on context and user behavior**. The travel interactions correctly identify **Amazon as the rainforest, Safari as wildlife, Java as the island, and Cruise as a ship journey**, while **"Safari download for Windows"** clearly indicates the browser sense.

For an industrial-scale recommendation system, WSD should combine **query context, user history, semantic embeddings, and click behavior** rather than relying on a single signal. This produces more accurate sense identification and consequently improves the relevance of travel recommendations.

Q4. Syntax-Driven Semantic Analysis in Legal Document Processing

1. Analyze How Syntactic Structure Influences Semantic-Role Assignment

Semantic roles describe the function that each entity performs in an event.

Common roles in these sentences are:

- **Agent** → entity performing the action
- **Object/Theme** → entity affected by the action
- **Recipient** → entity receiving something

The important point is that **grammatical subject and semantic agent are not always the same**. This becomes especially important in passive constructions.

Sentence 1: "The supplier delivered the equipment to the buyer."

Parse structure:

Subject → supplier
Verb → delivered
Object → equipment
PP → to the buyer

Semantic interpretation:

Entity	Role
Supplier	**Agent**
Equipment	**Object/Theme**
Buyer	**Recipient**

This interpretation is **correct**.

The supplier performs the action of delivering, the equipment is what is delivered, and the buyer receives it.

Deliver(Agent=Supplier, Object=Equipment, Recipient=Buyer)

Sentence 2: "The buyer received the equipment from the supplier."

The system gives:

Buyer = Agent
Equipment = Object
Supplier = Recipient

The first two assignments are reasonable, but **Supplier = Recipient is incorrect**.

The preposition **"from"** indicates the source of the equipment, not its recipient.

Correct semantic roles are:

Entity	Correct Role
Buyer	**Recipient**
Equipment	**Object/Theme**
Supplier	**Source**

Therefore:

Receive(Recipient=Buyer, Object=Equipment, Source=Supplier)

This is an important distinction because the grammatical subject **buyer** is not necessarily an Agent for the same type of event; the buyer is the entity receiving the equipment.

Sentence 3: "The company terminated the contract after the violation."

Parse:

```
Subject → company
Verb    → terminated
Object  → contract
PP      → after the violation
```

Semantic roles:

Entity	Role
Company	**Agent**
Contract	**Object/Theme**
Violation	**Temporal/Event information**

The supplied interpretation:

> Company = Agent, Contract = Object

is **correct**.

The phrase:

> "after the violation"

provides additional temporal/event information rather than changing the Agent/Object relationship.

Therefore:

Terminate(Agent=Company, Object=Contract)

Sentence 4: "The contract was terminated by the company."

This is a **passive construction**.

The system gives:

```
Contract = Agent
Company = Object
```

This is **incorrect**.

The grammatical subject is:

The contract

but the contract does **not** perform the termination.

The phrase:

"by the company"

identifies the entity performing the action.

Therefore:

Entity	**Correct Role**
Contract	**Object/Theme**
Company	**Agent**

Correct representation:

Terminate(Agent=Company, Object=Contract)

2. Identify the Incorrect Semantic-Role Assignments

There are **two clearly incorrect interpretations** in the supplied data.

Error 1 — Sentence 2

Given:

"The buyer received the equipment from the supplier."

System:

Buyer = Agent
Equipment = Object
Supplier = Recipient

The incorrect assignment is:

Supplier=Recipient

Correct:

Supplier=Source

because **"from the supplier"** identifies the source.

Error 2 — Sentence 4

Given:

"The contract was terminated by the company."

System:

Contract = Agent
Company = Object

Both assignments are reversed.

Correct:

Company=Agent Contract=Object

because the **company performs the termination**, while the **contract is affected by the action**.

Corrected Interpretation Table

Legal Sentence	Correct Semantic Roles
Supplier delivered equipment to buyer	Supplier = Agent; Equipment = Object; Buyer = Recipient
Buyer received equipment from supplier	Buyer = Recipient; Equipment = Object; Supplier = Source
Company terminated contract	Company = Agent; Contract = Object
Contract was terminated by company	Company = Agent; Contract = Object

3. Effect of Confusing Grammatical Subject with Semantic Agent

This is particularly dangerous in legal NLP.

A common mistake is:

> **Grammatical subject = Agent**

This works for many active sentences but fails for passive constructions.

Active voice

> **The company terminated the contract.**

Company → Subject + Agent
Contract → Object + Theme

Here:

Subject=Agent

Passive voice

> **The contract was terminated by the company.**

Contract → Subject + Theme
Company → Agent

Here:

Subject =Agent

Therefore, a legal NLP system that simply assumes:

Grammatical Subject=Agent

would incorrectly conclude:

Contract performed the termination.

That completely changes the legal meaning.

Why this matters in legal information extraction

Consider a system extracting:

```
Action: terminate
Agent: Contract
Object: Company
```

This could produce an entirely false legal record.

A correct extraction should be:

```
Action: terminate
Agent: Company
Object: Contract
```

Such errors can affect:

- Contract summaries
- Liability extraction
- Obligation identification
- Dispute analysis
- Compliance systems
- Legal search
- Automated contract review

Therefore, **syntactic structure must be analyzed before assigning semantic roles**.

4. Recommended Syntax-Driven Semantic Analysis Architecture

A robust legal NLP system should use a multi-stage syntax-driven architecture.

Proposed Architecture

```
Legal Document
    ↓
Sentence Segmentation
    ↓
Tokenization
    ↓
POS Tagging
    ↓
Syntactic Parsing
    ↓
Dependency / Constituency Analysis
```

↓
Voice Detection
↓
Semantic Role Labeling
↓
Prepositional Phrase Analysis
↓
Long-Distance Dependency Resolution
↓
Legal Event Representation
↓
Information Extraction

Step 1 – Syntactic Parsing

Identify:

- Subject
- Verb
- Object
- Prepositional phrases
- Clauses

For example:

The contract was terminated by the company.
↓
Passive construction

Step 2 – Voice Detection

The system should determine whether the sentence is:

- Active
- Passive

Active

Company → performed action

Passive

Company → introduced by "by"
Contract → grammatical subject

This prevents the system from automatically treating the subject as the Agent.

Step 3 – Semantic Role Labeling

Map syntactic constituents to semantic roles.

For:

> The supplier delivered the equipment to the buyer.

Supplier
↓
Agent

Equipment
↓

Object

Buyer
↓
Recipient

Step 4 – Prepositional Phrase Analysis

Prepositions can change the semantic role.

For example:

to the buyer

indicates a **Recipient**.

Whereas:

from the supplier

indicates a **Source**.

Therefore, the system should not ignore prepositional phrases

Step 5 – Long-Distance Dependency Handling

Legal sentences can contain clauses where the semantic relationship is separated by several words or clauses.

A dependency parser should track relationships across these structures rather than relying only on adjacent words.

Step 6 – Legal Event Representation

After semantic analysis, convert the sentence into a structured event.

For example:

EVENT: TERMINATE

Agent:
Company

Object:
Contract

Condition:
After the violation

This structured representation can then be used by downstream legal information-extraction systems.

Final Analysis

The data demonstrates that **syntax directly influences semantic interpretation**, but grammatical position alone cannot determine semantic roles.

The most important findings are:

1. **Sentence 1 is correctly interpreted.**
2. **Sentence 2 incorrectly labels the supplier as Recipient; it should be Source.**
3. **Sentence 3 is correctly interpreted.**
4. **Sentence 4 reverses the Agent and Object roles.**
5. Passive constructions demonstrate that **grammatical subject ≠ semantic Agent**.
6. A reliable legal NLP system should combine **syntactic parsing, voice detection, dependency analysis, semantic-role labeling, prepositional-phrase analysis, and long-distance dependency handling**.

Syntax→Semantic Roles→Legal Event Representation